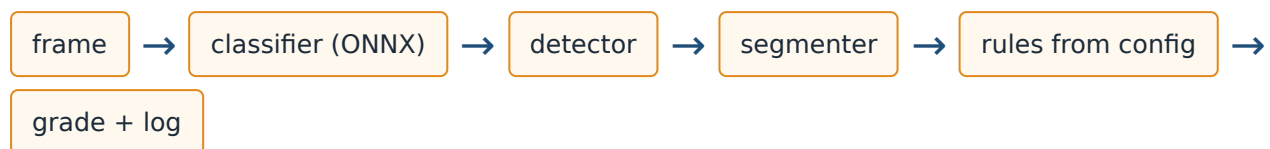


Tamr Vision: Run the Complete System on Google Colab

Computer Vision Systems Development (SDA-AIE-212), SDAIA Academy. Step-by-step guide for a first-time Colab user.

What you are about to run

Tamr Vision is the complete system from the course: a camera frame goes in, a grade and an eject signal come out, and every decision is logged. You built the parts in Labs 1 to 7. This package runs them together, once, on a free Google Colab machine, so you can see the whole thing work before you defend your own.



You need: a Google account, a browser, and the file `tamr_demo.zip` (about 45 MB). Nothing is installed on your computer.

Part A. Run it (about 10 minutes)

Step 1. Open Google Colab

Go to `colab.research.google.com` and sign in with your Google account.

You should see: a page with a dialog offering to open notebooks.

Step 2. Open the notebook

In the dialog choose the **Upload** tab, click **Browse**, and pick `Tamr_Vision_Colab.ipynb` from the unzipped package on your computer. (If the dialog is gone: File menu, Upload notebook.)

You should see: the notebook opens with a title "Tamr Vision: run the complete system" and seven numbered steps.

Step 3. Connect to a machine

Top right, click **Connect**. Wait until it shows a green tick with RAM and Disk bars. A CPU machine is enough; do not change the runtime type.

You should see: a green tick next to "RAM Disk" at the top right.

Step 4. Run Step 1: upload the package

Click inside the first code cell and press **Shift+Enter**. A **Choose Files** button appears under the cell. Click it and select `tamr_demo.zip`. The upload takes about a minute.

You should see: a printed list of files ending with `tamr_vision.py`.

Step 5. Run Step 2: install libraries

Shift+Enter on the next code cell. About two minutes. Orange warnings are normal.

You should see: a line starting with `torch` and three version numbers.

Step 6. Run Step 3: the system

Shift+Enter. This is the main run: it generates the data, exports the models to ONNX, checks parity, grades 80 frames, and writes the report. About three minutes.

You should see: five lines starting with `==== Step`, then a line `accuracy 0.8xx | p50 ... ms`, then `DONE`.

Step 7. Run Step 4: the report

Shift+Enter. The report appears below the cell. Scroll through it.

You should see: five number cards at the top, the flow diagram, the rules, latency and accuracy tables, and a gallery of frames with green or red borders.

Step 8. Run Step 5: one log line

Shift+Enter.

You should see: one JSON block with `decision`, `reason`, `raw_scores`, `latency_ms` and `versions`.

Step 9. Run Step 6: change the rules

Shift+Enter on the cell that edits the config and reruns. Then Shift+Enter on the cell that restores the config.

You should see: a new `decisions:` line with more rejects than in Step 6 of this guide.

Step 10. Run Step 7: download

Shift+Enter. Your browser downloads `report.html`, the INT8 ONNX classifier and the decision logs. Open `report.html` by double-clicking it.

You should see: the same report in a normal browser tab, without Colab.

Part B. What to look at in the report

Section	What it shows	Which lab it comes from
Number cards	frames graded, decision accuracy, p50 latency against the 380 ms budget, eject count, ONNX parity	Lab 7, Lab 8

1. Flow and rules	the five stages and the JSON config that holds every threshold; the code contains no numbers	Lab 8, Module 7
2. Deployment evidence	PyTorch versus ONNX max difference and decision flips; model sizes FP32 versus INT8; detector ONNX latency	Lab 7
3. Latency per stage	classify, detect, segment, decide at p50 and p99; the biggest number is where optimisation effort goes	Lab 7
4. Accuracy per session	s09 (evening) against s12 (daytime); the aggregate hides the weak slice	Lab 6
5. Confusion table	which true grades the system confuses; substandard graded as standard is the usual leak	Lab 6, Lab 8
6. Versions	classifier, detector, segmenter, dataset, rules and contract versions stamped on every frame	Module 7
7. Gallery	every frame with its boxes, decision, truth, reason, area and latency; red border means wrong	Lab 3, Lab 4, Lab 8

Part C. If something goes wrong

What you see	What to do
Step 1 says <code>No such file: tamr_demo.zip</code>	the upload did not finish or you chose a different file; run the cell again and pick <code>tamr_demo.zip</code>
Step 2 prints a red ERROR line	run the cell once more; Colab package servers sometimes time out
Step 3 stops with <code>AssertionError: ONNX output differs</code>	this should not happen; tell the instructor, it means the export is broken on this machine
the page says "Runtime disconnected"	Colab idled out; click Connect, then rerun from Step 1 (uploads are lost on disconnect)
Step 4 shows nothing	run Step 3 first; the report is written at the end of it
everything is slow	normal on a free CPU; Step 3 can take up to five minutes

Part D. For your own capstone

Your capstone is this package with your own pieces swapped in: your Lab 2 classifier checkpoint in `checkpoints/`, your Lab 3 and Lab 4 YOLO weights, your operating points and thresholds in `config/grading_rules.json`, and your v2 dataset release. Nothing else changes. If `run_demo.py` still prints DONE and the report still renders, your system is assembled.